

**WORKING WITH DATA ANALYSIS
EXPRESSIONS (DAX) IN MS POWER
BI.
DAX FUNCTIONS.**

DAX overview

Data Analysis Expressions (DAX) is a formula expression language used in Analysis Services, Power BI, and Power Pivot in Excel. DAX formulas include functions, operators, and values to perform advanced calculations and queries on data in related tables and columns in tabular data models.

DAX includes a library of over 200 functions, operators, and constructs. Its library provides immense flexibility in creating measures to calculate results for just about any data analysis need.

DAX formulas are a lot like Excel formulas. DAX even has many of the same functions as Excel, such as DATE, SUM, and LEFT. But the DAX functions are mean to work with relational data like we have in Power BI Desktop.

Calculations

DAX formulas are used in measures, calculated columns, calculated tables, and row-level security.

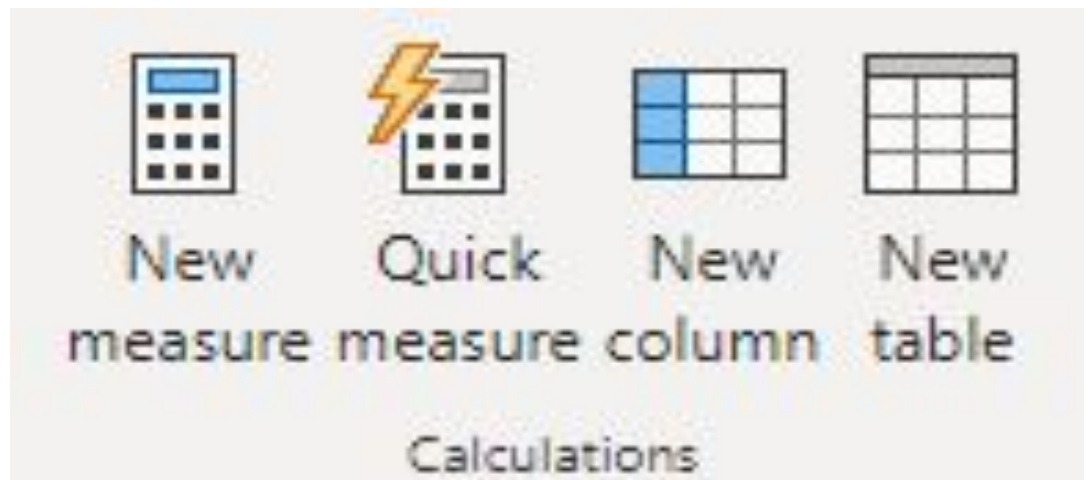
OBJECTS CREATED USING DAX

With DAX you can create

- Measures
- Columns
- Calculated Tables

The rule:

For all measures and calculations, use DAX MEASURES wherever possible.



Measures and calculated columns

- ❑ A **measure** is a basic concept in DAX and is an expression that allows you to calculate the required indicator based on data from the model. An example of using measures is calculating the average, sum, number of unique records, and so on.
- ❑ The **calculated column** also allows you to calculate metrics, but the calculation is performed for each row of the table separately and the result is saved in a separate field (a new table column).
- ❑ Measures, unlike columns, are calculated only when using the visualization, while the values of a new column are calculated immediately after it is created.
- ❑ The measure is not physically displayed in the table and the model, and does not weigh down the model.
- ❑ The calculated column is stored in the model.

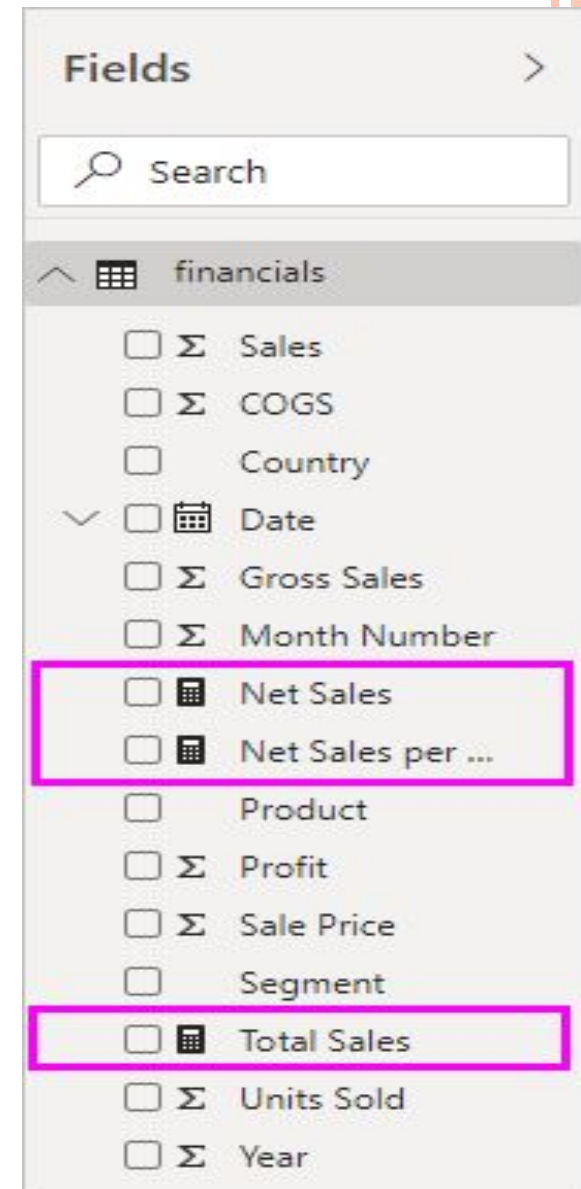


Measures

In Power BI Desktop, measures are created and displayed in Report View or Data View.

Measures are dynamic calculation formulas where the results change depending on context. Measures are used in reporting that support combining and filtering model data by using multiple attributes such as a Power BI report or Excel PivotTable or PivotChart. Measures are created by using the DAX formula bar in the model designer.

Measures you create yourself appear in the **Fields** list with a calculator icon and available to all users of the model. You can name measures whatever you want and add them to a new or existing visualization just like any other field.



Measures

A formula in a measure can use standard aggregation functions automatically created by using the Autosum feature, such as COUNT or SUM, or you can define your own formula by using the DAX formula bar. Named measures can be passed as an argument to other measures.

For example, this is a very simple measure formula:

Total Sales = SUM([Sales Amount])

Unlike calculated columns, the syntax for a measure includes the measure's name preceding the formula.

Next example of using the DAX to create the measure

Projected Sales = SUM('Sales'[Last Years Sales])*1.06



ORGANIZING YOUR MEASURES

Measures have a Home table that defines where they're found in the field list. You can change their location by choosing a location from the tables in your model.



You can create a special table that contains only measures. That table always appears at the top of the Fields. To do so, create a table with just one column. You can use Enter Data to create that table. Then move your measures to that table. Finally, hide the column, but not the table, that you created. Select the arrow at the top of Fields to close and reopen the fields list to see your changes.



USING QUICK MEASURES

To quickly and easily perform standard and resource-intensive calculations, you can use quick measures.

Quick Measure executes a set of DAX commands in the background, and then presents the results for use in a report.

No need to write DAX code — everything is done automatically based on the input data.



USING QUICK MEASURES

Five types of fast measure calculations with corresponding calculations.

Aggregate by category:

- Average per category
- Variance by category
- Maximum per category
- Minimum per category
- Weighted average by category

Filters:

- Filtered value
- Difference with filtered value
- Percentage of the difference with the filtered value
- Sales from new customers

Logic of operations with time

- Total since the beginning of the year
- Total since the beginning of the quarter
- Total from the beginning of the month
- Change by year
- Change by quarter

Month 1



USING QUICK MEASURES

Quick measures

Calculation

Average per category ▼

Calculate the average of the base value within the category. [Learn more](#)

Base value ⓘ

1 Sales ×

Category ⓘ

Category ×

Fields

Search

- ▼  Budget
- ▼  Calendar
- ▼  Customer
- ^  Product
 -  20. Selected Product
 - Category
 - Color
 - Σ DaysToManufacture
 - ListPrice
 - ModelName
 - ProductDescription

1 1 Sales average per Category =

2 `AVERAGEX(KEEPFILTERS(VALUE('Product'[Category])), CALCULATE([1 Sales]))`

TABLE OF MEASURES

It is convenient to create a measure table when there are many measures in different tables.

Steps for creating a measure table:

- ❑ Home tab-Enter Data, renaming the table name to MeasureTable-Load.
- ❑ Create a new test measure measure = “Test”
- ❑ Delete column Column1
- ❑ Save the project and close it
- ❑ Re-open the project, the resulting table of measures will be located at the beginning of the list of tables
- ❑ Select each measure and select MeasureTable as the Home table for the measure.

You can create separate tables for measures (for measures for sales, products, customers, etc.).)



VARIABLES

Use variables to

- avoid repeating the same expressions in calculations
- don't create many measures

NewMeasure =

VAR

TotalQuantity = SUM (Sales[Quantity])

RETURN

IF (TotalQuantity > 1000;

TotalQuantity * 0.95;

TotalQuantity * 1.25)



DAX RESOURCES

- <https://dax.guide/>
- <https://community.powerbi.com/t5/Data-Stories-Gallery/DAX-Cheat-Sheet-Success-of-CSS-Selector/td-p/559474> и др.

Microsoft Power BI Community > Galleries > Data Stories Gallery > DAX Cheat Sheet - Success of CSS Selector

DAX Cheat Sheet

Iron Man's Lab

Perfect the world with BI

Search by Function Name

FunctionGroupName

- ☐ Date and Time Functions (DAX)
- ☐ Filter Functions (DAX)
- ☐ Information Functions (DAX)
- ☐ Logical Functions (DAX)
- ☐ Math and Trig Functions (DAX)
- ☐ Other Functions (DAX)
- ☐ Parent and Child Functions (DAX)
- ☐ Statistical Functions (DAX)
- ☒ Text Functions (DAX)
 - ☐ BLANK Function (DAX)
 - ☐ CODE Function (DAX)
 - ☐ CONCATENATE Function (DAX)
 - ☐ CONCATENATEX Function (DAX)
 - ☐ EXACT Function (DAX)
 - ☐ FIND Function (DAX)
 - ☐ FIXED Function (DAX)
 - ☐ FORMAT Function (DAX)

Function Name	Function Description	Syntax	Return Value	URL Link
BLANK Function (DAX)	Returns a blank.	BLANK()	A blank.	Link
CODE Function (DAX)	Returns a numeric code for the first character in a text string. The returned code corresponds to the character set used by your computer.	CODE(text)	A numeric code for the first character in a text string.	Link
CONCATENATE Function (DAX)	Joins two text strings into one text string.	CONCATENATE(<text1>, <text2>)	The concatenated string.	Link
CONCATENATEX Function (DAX)	Concatenates the result of an expression evaluated for each row in a table.	CONCATENATEX(<table>, <expression>, [delimiter])	A text string.	Link
EXACT Function (DAX)	Compares two text strings and returns TRUE if they are exactly the same, FALSE otherwise. EXACT is case-sensitive but ignores formatting differences. You can use EXACT to test text being entered into a document.	EXACT(<text1>, <text2>)	True or false. (Boolean)	Link
FIND Function (DAX)	Returns the starting position of one text string within another text string. FIND is case-sensitive.	FIND(<find_text>, <within_text>[, <start_num>[, <NotFoundValue>]])	Number that shows the starting point of the text string you want to find.	Link
FIXED Function (DAX)	Rounds a number to the specified number of decimals and returns the result as text. You can specify that the result be returned with or without commas.	FIXED(<number>, <decimals>, <no_commas>)	A number represented as text.	Link
FORMAT Function (DAX)	Converts a value to text according to the specified format.	FORMAT(<value>, <format_string>)	A string containing value formatted as defined	Link

DAX PRACTICE

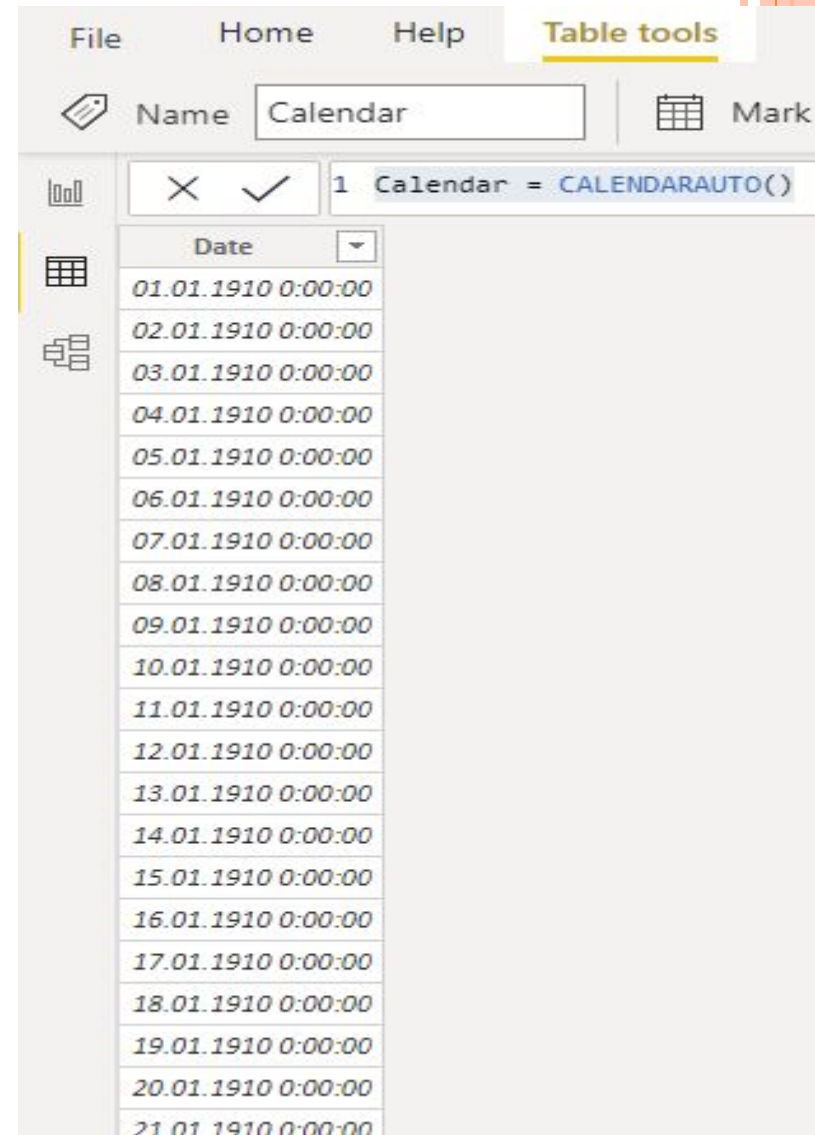
Creating a table (Calendar)

1. DAX formula

Calendar = CALENDARAUTO()

automatically creates a table from the model, but does not always create the correct table.

It is advisable to use the CALENDAR() function.



The screenshot shows the Microsoft Excel interface with the 'Table tools' ribbon selected. A table named 'Calendar' is displayed, created using the formula `1 Calendar = CALENDARAUTO()`. The table contains a single column named 'Date' with the following values:

Date
01.01.1910 0:00:00
02.01.1910 0:00:00
03.01.1910 0:00:00
04.01.1910 0:00:00
05.01.1910 0:00:00
06.01.1910 0:00:00
07.01.1910 0:00:00
08.01.1910 0:00:00
09.01.1910 0:00:00
10.01.1910 0:00:00
11.01.1910 0:00:00
12.01.1910 0:00:00
13.01.1910 0:00:00
14.01.1910 0:00:00
15.01.1910 0:00:00
16.01.1910 0:00:00
17.01.1910 0:00:00
18.01.1910 0:00:00
19.01.1910 0:00:00
20.01.1910 0:00:00
21.01.1910 0:00:00

DAX PRACTICE

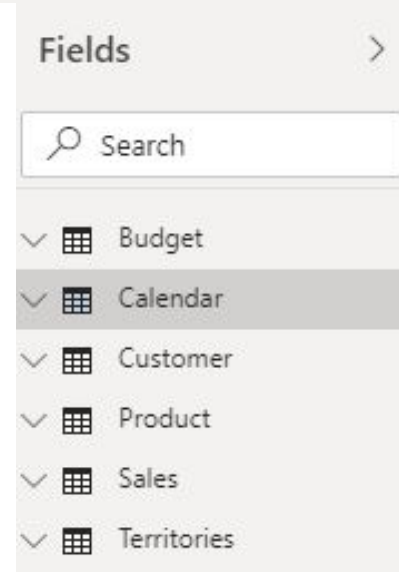
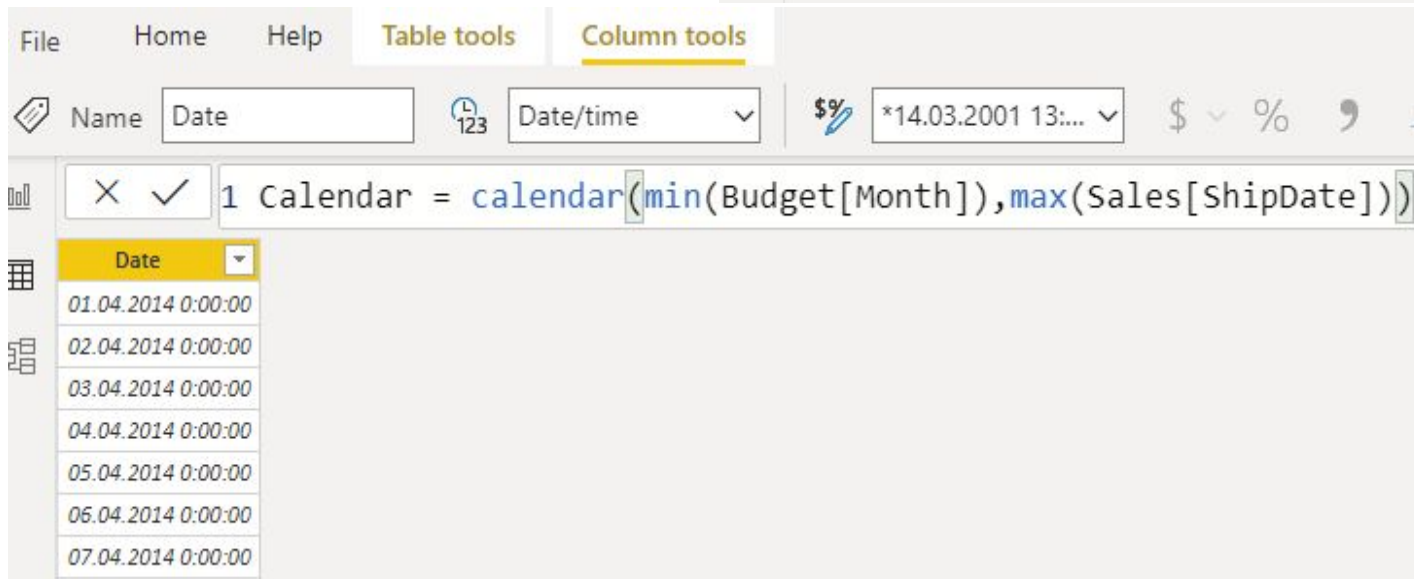
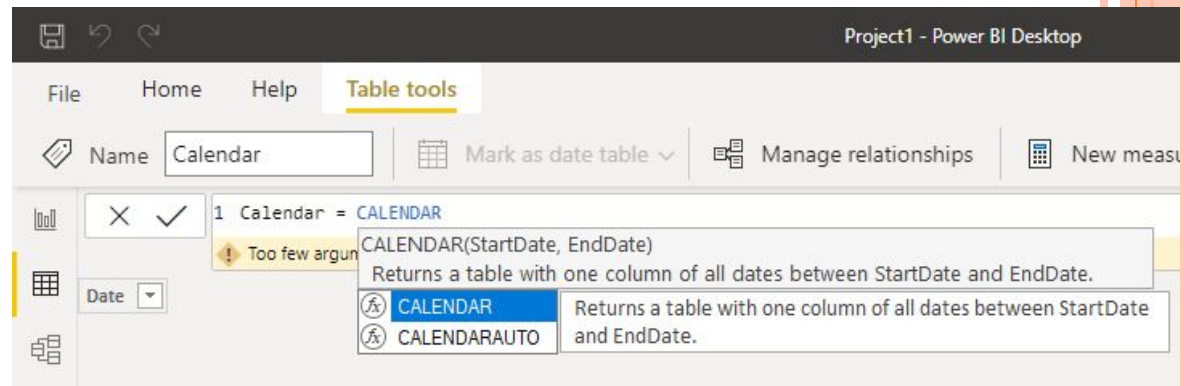
Creating a table (Calendar)

2. The DAX formula `CALENDAR ()` creates a single-column table with data between Startdate, Enddate.

Dates: 01.04.2014

– 07.01.2017,

1103 lines



DAX PRACTICE

Creating a table (Calendar)

In DAX formulas, the default delimiters are (,), and (;) can be used. The last brackets are automatically supplemented, and the user is responsible for the middle brackets.

Options

GLOBAL

- Data Load
- Power Query Editor
- DirectQuery
- R scripting
- Python scripting
- Security
- Privacy
- Regional Settings
- Updates
- Usage Data
- Diagnostics
- Preview features
- Auto recovery
- Report settings

CURRENT FILE

Application language

Language used in the Power BI Desktop user interface, such as the ribbon and dialog boxes.

English (United States) ▾

Model language

Language used when comparing strings in data and for creating internal date fields. This only applies when a report is first created, and can't be changed on existing reports.

English (United States) ▾

DAX separators

Specify the culture to use for list and decimal separators in DAX expressions:

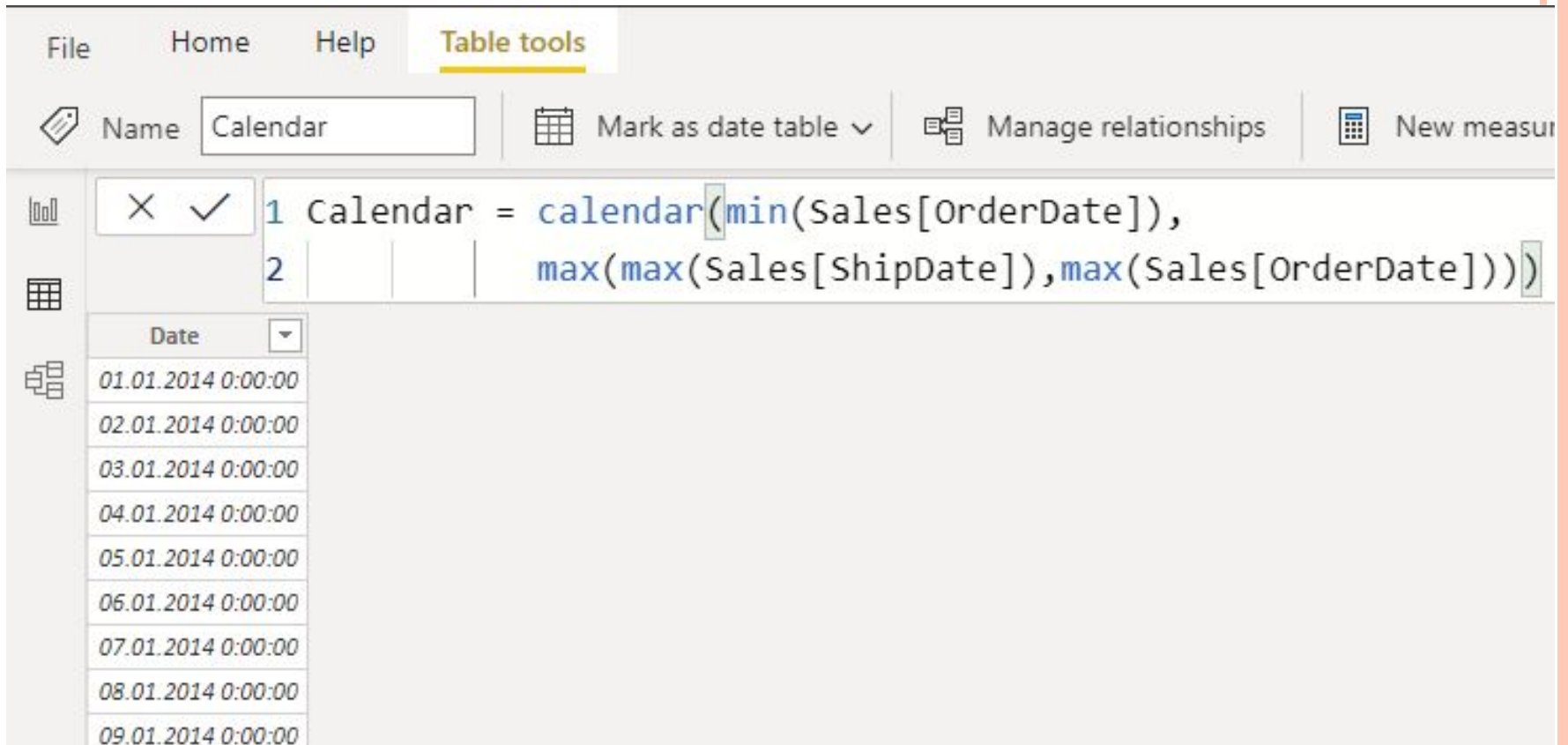
- ☒ (Recommended) Use standard DAX separators: comma (,) as list separator and dot (.) as decimal separator
- ☐ Use localized DAX separators: list and decimal separators are defined by Windows locale settings

[Learn more](#)

DAX PRACTICE

Creating a table (Calendar)

Changing the formula. If you change the date of the shipment or order, the Calendar Table data will change.



The screenshot shows the Microsoft Excel interface with the 'Table tools' ribbon active. The 'Name' box contains 'Calendar'. The 'Mark as date table' button is visible. The formula bar shows the following DAX formula:

```
1 Calendar = calendar(min(Sales[OrderDate]),  
2 max(max(Sales[ShipDate]),max(Sales[OrderDate])))
```

Below the formula bar, a table is displayed with a column header 'Date' and a dropdown arrow. The table contains the following dates:

Date
01.01.2014 0:00:00
02.01.2014 0:00:00
03.01.2014 0:00:00
04.01.2014 0:00:00
05.01.2014 0:00:00
06.01.2014 0:00:00
07.01.2014 0:00:00
08.01.2014 0:00:00
09.01.2014 0:00:00

DAX PRACTICE

Creating a table (Calendar)

Changing the formula. For the calendar data to start from 01.01. and end on 31.12, you need to add the Date and Year functions.

The screenshot shows the Microsoft Power BI interface. The top ribbon has tabs for File, Home, Help, and Table tools. The Table tools tab is active, showing options like Name (Calendar), Mark as date table, Manage relationships, New measure, and Quick measure. The DAX formula bar contains the following formula:

```
1 Calendar = Calendar(date(year(min(Sales[OrderDate])),1,1),  
2 date(year(max(max(Sales[ShipDate])),max(Sales[OrderDate]))),12,31))
```

Below the formula bar, a preview of the Calendar table is shown. The table has a single column named 'Date'. The first four rows show dates from 01.01.2014 0:00:00 to 04.01.2014 0:00:00. The last seven rows show dates from 25.12.2017 0:00:00 to 31.12.2017 0:00:00.

Date
01.01.2014 0:00:00
02.01.2014 0:00:00
03.01.2014 0:00:00
04.01.2014 0:00:00
...
25.12.2017 0:00:00
26.12.2017 0:00:00
27.12.2017 0:00:00
28.12.2017 0:00:00
29.12.2017 0:00:00
30.12.2017 0:00:00
31.12.2017 0:00:00

Table: Calendar (1,461 rows)

DAX PRACTICE

Creating a table (Calendar)

Adding columns. Table tools- New Column

```
1 Year = Year('Calendar'[Date])
```

```
1 MonthNum = MONTH('Calendar'[Date])
```

```
1 MonthNum1 = 'Calendar'[Date].[MonthNo]
```

```
1 Month = 'Calendar'[Date].[Month]
```

```
1 YearMonth = Format('Calendar'[Date], "yyyy-mm")
```

```
1 Quarter = 'Calendar'[Date].[Quarter]
```

```
1 WeekDayNu = WEEKDAY('Calendar'[Date], 2)
```



DAX PRACTICE

Creating a table (Calendar)

Adding columns .Table tools- New Column

FileHomeHelpTable toolsColumn tools

NameCalendar

Mark as date table

Manage relationships

New column

1 WeekDayNu = WEEKDAY('Calendar'[Date],2)

Date	Year	MonthNum	MonthNum1	Month	YearMonth	Quarter	WeekDayNu
01.01.2014 0:00:00	2014	1	1	January	2014-01	Qtr 1	3
02.01.2014 0:00:00	2014	1	1	January	2014-01	Qtr 1	4
03.01.2014 0:00:00	2014	1	1	January	2014-01	Qtr 1	5
04.01.2014 0:00:00	2014	1	1	January	2014-01	Qtr 1	6
05.01.2014 0:00:00	2014	1	1	January	2014-01	Qtr 1	7
06.01.2014 0:00:00	2014	1	1	January	2014-01	Qtr 1	1
07.01.2014 0:00:00	2014	1	1	January	2014-01	Qtr 1	2
08.01.2014 0:00:00	2014	1	1	January	2014-01	Qtr 1	3
09.01.2014 0:00:00	2014	1	1	January	2014-01	Qtr 1	4
10.01.2014 0:00:00	2014	1	1	January	2014-01	Qtr 1	5
11.01.2014 0:00:00	2014	1	1	January	2014-01	Qtr 1	6
12.01.2014 0:00:00	2014	1	1	January	2014-01	Qtr 1	7
13.01.2014 0:00:00	2014	1	1	January	2014-01	Qtr 1	1
14.01.2014 0:00:00	2014	1	1	January	2014-01	Qtr 1	2
15.01.2014 0:00:00	2014	1	1	January	2014-01	Qtr 1	3
16.01.2014 0:00:00	2014	1	1	January	2014-01	Qtr 1	4
17.01.2014 0:00:00	2014	1	1	January	2014-01	Qtr 1	5

DAX PRACTICE

Types of weekdays of the WeekDay function

Project1 - Power BI Desktop

Gullbakyt Sembina

File Home Help Table tools Column tools

Name Calendar Mark as date table Manage relationships New measure Quick measure New column New table

1 WeekDay=WEEKDAY('Calendar'[Date],1)

WEEKDAY(Date, [ReturnType])
Returns a number from 1 to 7 identifying the day of the week of a date.

1 Sunday=1 through Saturday=7

2 Monday=1 through Sunday=7

3 Monday=0 through Sunday=6

Fields

Budget

Date	Year	Month	Day	WeekDay
01.01.2014 0:00:00	2014	1	1	1
02.01.2014 0:00:00	2014	1	2	2
03.01.2014 0:00:00	2014	1	3	3

DAX PRACTICE

Adding columns. Logical functions

SWITCH, IF

```
1 WorkDay = SWITCH('Calendar'[WeekDayNu],6,"WeekEnd", 7, "WeekEnd", "WorkDay")
```

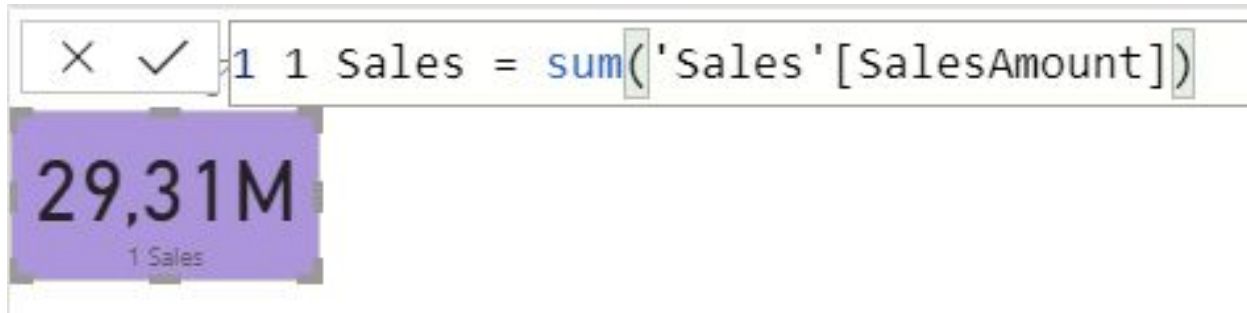
```
1 WeekDay = IF('Calendar'[WeekDayNu]=1,"Mon", IF('Calendar'[WeekDayNu]=2, "Tue",  
2 IF('Calendar'[WeekDayNu]=3, "Wed", IF('Calendar'[WeekDayNu]=4, "Thu",  
3 IF('Calendar'[WeekDayNu]=5,"Fri", IF('Calendar'[WeekDayNu]=6,"Sat", "Sun"))))))
```

WorkDay	WeekDay
WorkDay	Wed
WorkDay	Thu
WorkDay	Fri
WeekEnd	Sat
WeekEnd	Sun
WorkDay	Mon
WorkDay	Tue
WorkDay	Wed
WorkDay	Thu
WorkDay	Fri
WeekEnd	Sat
WeekEnd	Sun
WorkDay	Mon
WorkDay	Tue
WorkDay	Wed
WorkDay	Thu
WorkDay	Fri
WeekEnd	Sat
WeekEnd	Sun
WorkDay	Mon
WorkDay	Tue



DAX PRACTICE

Creating simple measures. Tab Data – Table Tools - New Measure or Popup Menu - New Measure



The screenshot shows the DAX formula bar with the formula `1 1 Sales = sum('Sales'[SalesAmount])`. Below the formula bar, a purple result card displays the value **29,31M** with the label **1 Sales** underneath.



The screenshot shows the Power BI ribbon with the 'Measure tools' tab selected. The ribbon includes a 'Name' field with '1 Sales', a 'Home table' dropdown set to 'Sales', a 'Format' dropdown set to 'General', and a 'Uncategorized' dropdown. There are also icons for currency, percentage, comma, decimal, and auto formatting.



The screenshot shows the DAX formula bar with the formula `1 2Min Price = Min(Sales[UnitPrice])`. Below the formula bar, there are two result cards: a purple card displaying **29,31M** with the label **1 Sales**, and a blue card displaying **0,57** with the label **2Min Price**.



DAX PRACTICE

Creating simple measures.

1 3 Max Sales = **Max**(Sales[SalesAmount])

29,31M
1 Sales

0,57
2 Min Price

3,58K
3 Max Sales

1 4 AverageSales = **AVERAGE**(Sales[SalesAmount])

503,67
4 AverageSales

1 5 CustomerCount = **COUNT**(Sales[CustomerKey])

58K
5 CustomerCount

18K
6 CustomerCount1



DAX PRACTICE

Creating simple measures.

```
1 6 CustomerCount1 = DISTINCTCOUNT(Sales[CustomerKey])
```

58K

5 CustomerCount

18K

6 CustomerCount1

✕ ✓

```
1 7 SalesA = sumx(Sales, Sales[OrderQuantity]*Sales[UnitPrice])
```

29 308K

29 308K

7 SalesA

✕ ✓

```
1 8 Difference = [1 Sales] - [7 SalesA]
```

6,45%

8 Difference



DAX PRACTICE

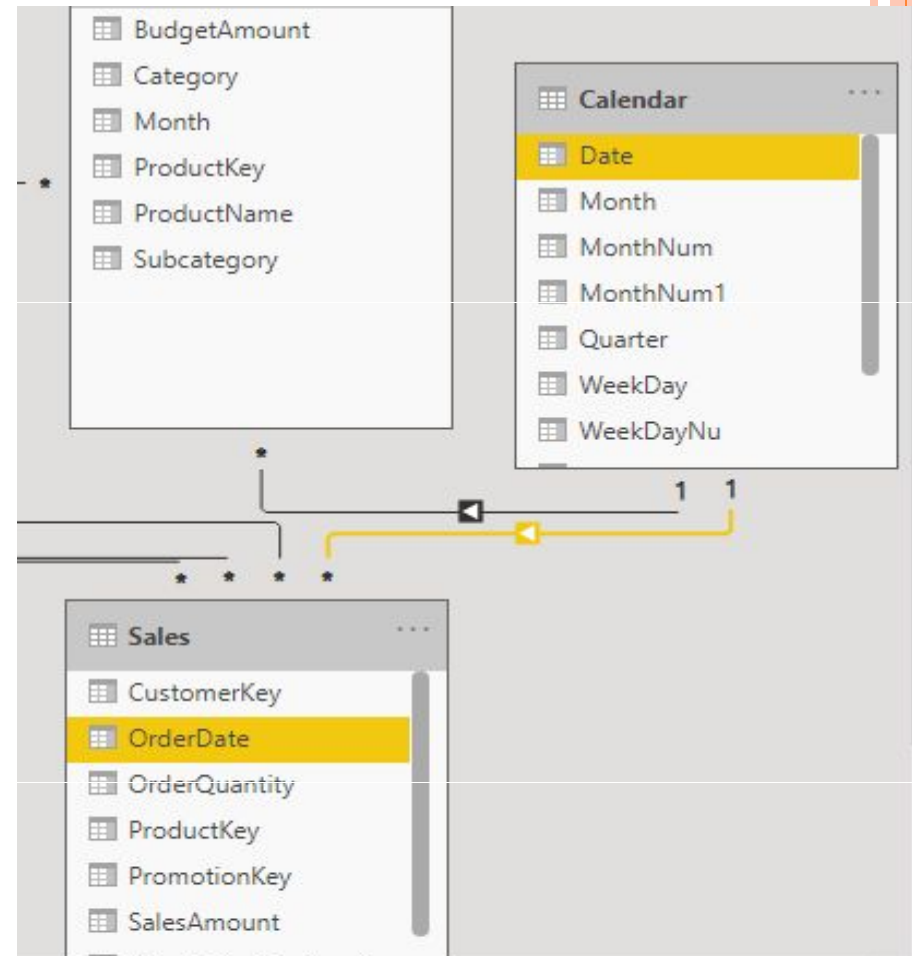
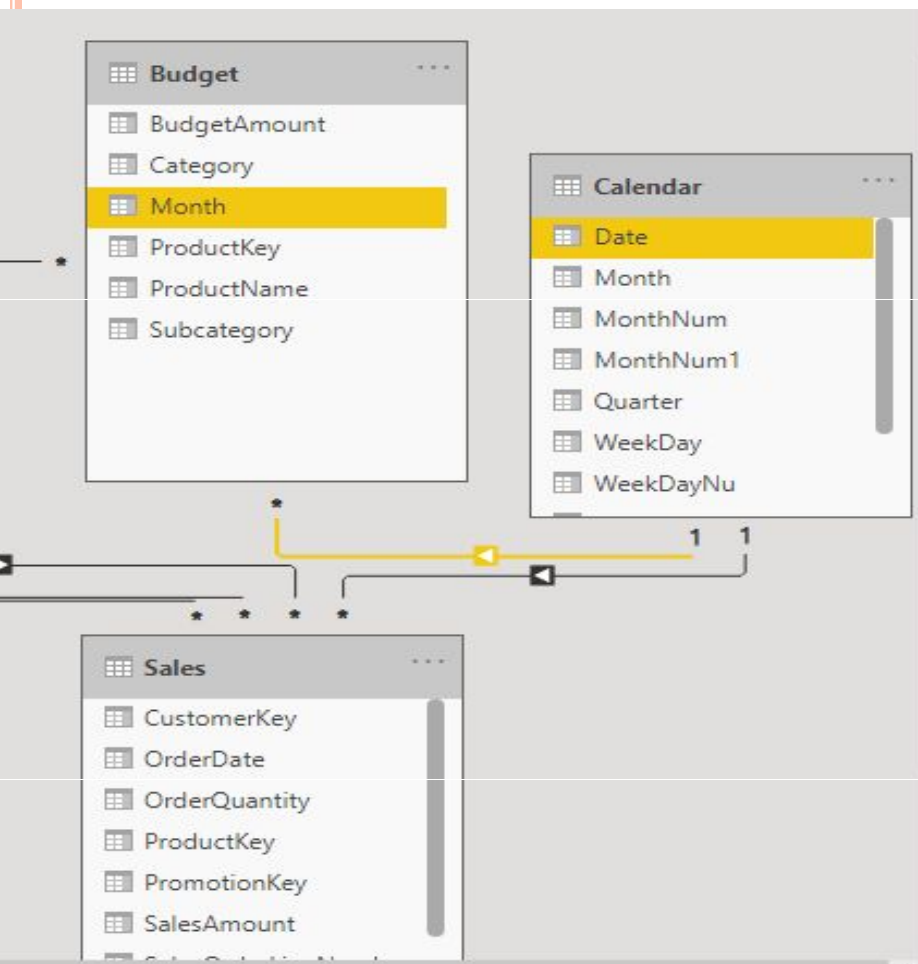
Creating complex measures. Variables in measures

```
1 9 D1 =  
2 VAR Sales1 = sum('Sales'[SalesAmount])  
3 VAR Max_Sales = Max(Sales[SalesAmount])  
4 VAR CustomerCount = DISTINCTCOUNT(Sales[CustomerKey])  
5 VAR SalesA = sumx(Sales, Sales[OrderQuantity]*Sales[UnitPrice])  
6 Return Sales1
```

Since the measures are arranged alphabetically in the columns of tables, it is convenient to number them so that they appear at the beginning of the list.



TABLE LINKS CALENDAR



ITERATOR FUNCTIONS

- ❑ The iterator functions provide flexibility and control over the summation of data in model calculations.
- ❑ The iterator functions are SUMX, COUNTX, MINX, MAXX, and others. In addition, there are specialized iterator functions that perform filtering, ranking, semi-additive calculations in time, and much more.
- ❑ The main characteristic of all iterator functions is that you must pass a table and an expression.

Syntax: **SUMX(<table>, <expression>)**

- ❑ A table can be a reference to a model table, or an expression that returns a table object. The evaluation of the expression must give a scalar value.



ITERATOR FUNCTIONS

- The iterator functions list all the rows in each table and evaluate the expression for each row.

```
1 Sum of Margin = SUMX(  
2     FactInternetSales,  
3     FactInternetSales[SalesAmount]-FactInternetSales[TotalProductCost]  
4 )
```

TotalProductCost	SalesAmount	[SalesAmount]-[TotalProductCost]
26.1763	69.99	43.8137
13.09	35	21.91
11.2163	29.99	18.7737
6.9223	8.99	2.0677
6.9223	8.99	2.0677
9.1593	24.49	15.3307
6.9223	8.99	2.0677
419.7784	769.49	349.7116
755.1508	1214.85	459.6992
419.7784	769.49	349.7116
8.2205	21.98	13.7595
11.2163	29.99	18.7737
11.2163	29.99	18.7737
308.2179	564.99	256.7721
308.2179	564.99	256.7721
12.1924	32.6	20.4076
6.9223	8.99	2.0677
13.0863	34.99	21.9037
3.7363	9.99	6.2537
6.9223	8.99	2.0677
12.1924	32.6	20.4076
11.2163	29.99	18.7737
9.1593	24.49	15.3307
419.7784	769.49	349.7116
6.9223	8.99	2.0677
13.0863	34.99	21.9037
12.1924	32.6	20.4076
26.1763	69.99	43.8137
419.7784	769.49	349.7116
6.9223	8.99	2.0677
3.7363	9.99	6.2537

Calculates the expression ONE ROW AT A TIME, and stores the result in TEMPORARY memory

ITERATOR FUNCTIONS

- The iterator functions list all the rows in each table and evaluate the expression for each row.

TotalProductCost	SalesAmount	[SalesAmount]-[TotalProductCost]			
26.1763	69.99	43.8137			
13.09	35	21.91			
11.2163	29.99	18.7737			
6.9223	8.99	2.0677			
6.9223	8.99	2.0677			
9.1593	24.49	15.3307			
6.9223	8.99	2.0677			
419.7784	769.49	349.7116			
755.1508	1214.85	459.6992			
419.7784	769.49	349.7116			
8.2205	21.98	13.7595			
11.2163	29.99	18.7737			
11.2163	29.99	18.7737			
308.2179	564.99	256.7721			
308.2179	564.99	256.7721			
12.1924	32.6	20.4076			
6.9223	8.99	2.0677			
13.0863	34.99	21.9037			
3.7363	9.99	6.2537			
6.9223	8.99	2.0677			
12.1924	32.6	20.4076			
11.2163	29.99	18.7737			
9.1593	24.49	15.3307			
419.7784	769.49	349.7116			

SUM of all values will be calculated, and then temporary memory will be released

Important!

- When using iterator functions, make sure that you do not use large tables (rows) with expressions that use advanced DAX functions.
- Some functions, such as the **SEARCH** (DAX) function, which scans a text value for specific characters or text, can lead to poor performance.



COMPLEX SUMMARY DATA GENERATION

The file was first downloaded and opened **Adventure Works DW 2020 M05.pbix**.

Then the following DAX measure is added

1. Revenue = SUMX(Sales, Sales[Order Quantity] * Sales[Unit Price] * (1 - Sales[Unit Price Discount Pct]))

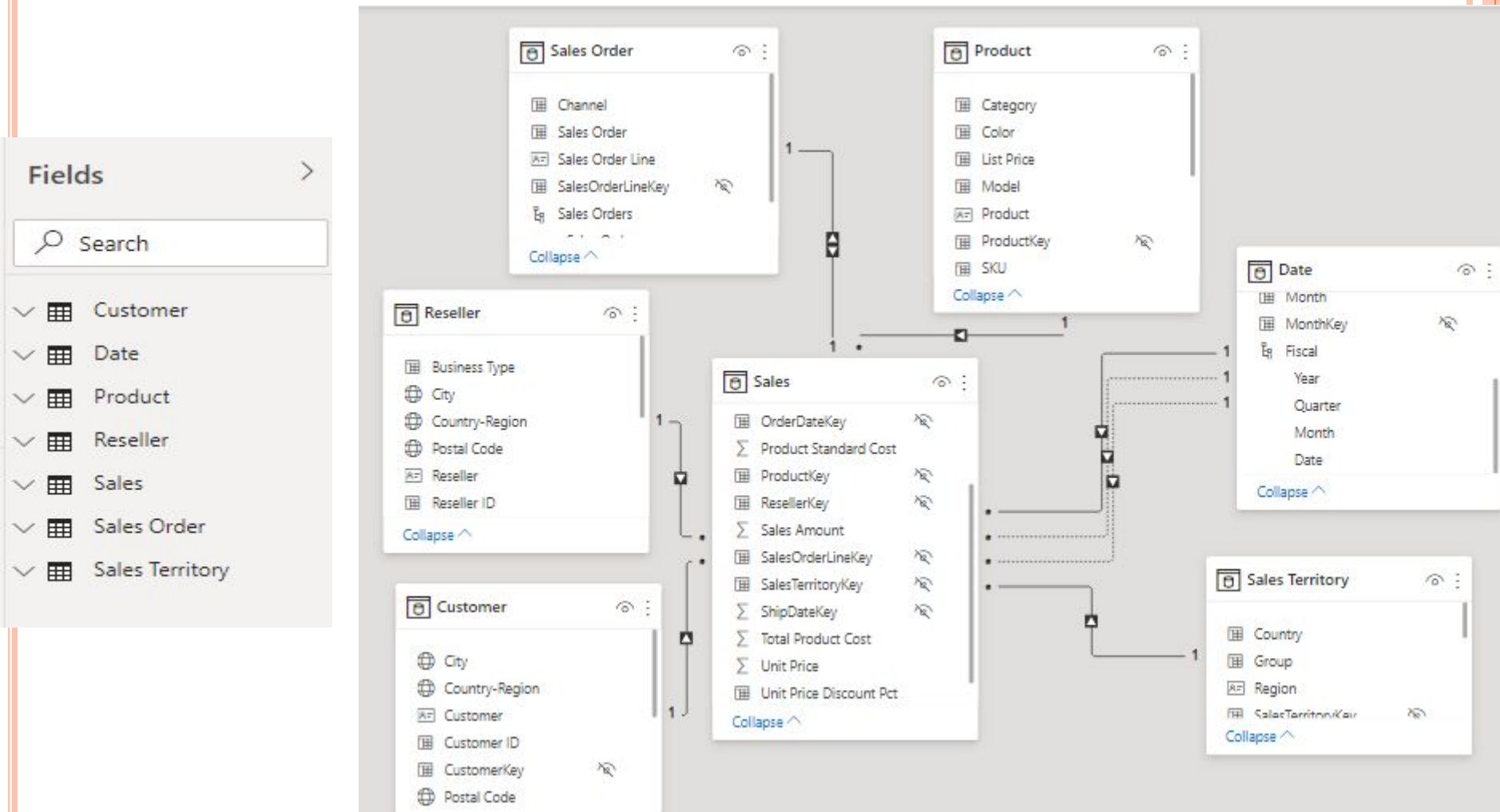
Format the **Revenue** measure as a currency with two decimal places, and then add it to the table visual that contains the Month column data from the Date table.

Using the iterator function, the **Revenue** measure formula aggregates more than the values of a single column. For each row, it uses the three-column row context values to get the revenue amount. **Row context = current row**

The row context is a key component of calculated columns.

PROJECT STRUCTURE

Adventure Works DW 2020 M05.pbix



COMPLEX SUMMARY DATA GENERATION

Now add another DAX measure

```
2.Discount = SUMX( Sales, Sales[Order Quantity] *  
( RELATED('Product'[List Price]) - Sales[Unit Price] ) )
```

Format the **Discount** measure as a currency with two decimal places, and then add it to the table visual.

The formula uses the **RELATED** function.

Remember that the row context does not extend beyond the table. If your formula needs to refer to columns in other tables and there are relationships between the tables in the model, use the **RELATED** function to link.

RELATED function>Returns a related value from another table. **RELATED(<column>)** syntax)

GENERATING SUMMARY DATA WITH A HIGHER LEVEL OF DETAIL

The following example describes the requirement to get a report on average income.

Add the following DAX measure

3.Revenue Avg Order Line= AVERAGEX(Sales,
Sales[Order Quantity] * Sales[Unit Price] * (1 - Sales[Unit
Price Discount Pct]))

Revenue Avg Order Line - the average value of revenue per order line.

Format the measure, and then add it to the visual element of the table.



GENERATING SUMMARY DATA WITH A HIGHER LEVEL OF DETAIL

In the following example, the iterator function is used to create a measure that increases the granularity to the level of a sales order (a sales order consists of one or more order lines).

Add the following DAX measure

4. Revenue Avg Order = AVERAGEX(VALUES ('Sales Order'[Sales Order]), [1.Revenue])

- The formula uses the **VALUES** (DAX) function. With this function, formulas can determine which values are in the filter context. In this case, the AVERAGEX function iterates over each sales order in the filter context.
- In other words, it iterates through each sales order in a month.
- **Filter context is a set of filters in a pivot table**

VISUALIZATION WITH ALL MEASURES

Month	Revenue	Discount	Revenue Avg Order Line	Revenue Avg Order
2017 Jul	\$1,423,357.32	\$326,219.06	\$2,220.53	\$4,352.77
2017 Aug	\$2,057,902.45	\$1,031,506.86	\$2,179.98	\$8,794.45
2017 Sep	\$2,523,947.55	\$1,342,355.67	\$2,014.32	\$9,670.30
2017 Oct	\$561,681.48	\$0.00	\$3,228.05	\$3,228.05
2017 Nov	\$4,764,920.16	\$2,692,205.61	\$2,227.64	\$12,441.04
2017 Dec	\$596,746.56	\$0.00	\$3,174.18	\$3,174.18
2018 Jan	\$1,327,674.63	\$475,813.07	\$2,329.25	\$5,698.17
2018 Feb	\$3,936,463.31	\$2,237,412.77	\$2,321.03	\$12,301.45
2018 Mar	\$700,873.18	\$0.00	\$3,200.33	\$3,200.33
2018 Apr	\$1,519,275.24	\$588,995.26	\$2,182.87	\$6,356.80
2018 May	\$2,960,378.09	\$1,514,891.49	\$2,219.17	\$9,642.93
2018 Jun	\$1,487,671.19	\$1,659,893.12	\$1,398.19	\$4,752.94

As expected, the average revenue for an order is always higher than the average revenue for an individual order line.

FUNCTIONS OF THE LOGIC OPERATIONS WITH TIME

Date table requirement

To work with the DAX time logic functions, you must meet the requirements for a mandatory model, for example, the model must have at least one date table. A date table is a table that meets the requirements below.

- ❑ It must contain a Date (or date/time) data type column, known as a *date column*.
- ❑ The date column must contain unique values.
- ❑ The date column must not contain EMPTY VALUES.
- ❑ There should be no missing dates in the date column.
- ❑ The date column should cover the full years. The year does not have to be a calendar year (January–December).
- ❑ The date table should be displayed as a table with dates.

METHODS FOR GENERATING SUMMARY DATA BY TIME

Function DAX DATESYTD — returns a single-column table that contains the dates from the beginning of the year (YTD) for the current filter context. This group also includes functions

DAX DATESMTD and DATESQTD from the beginning of the month (MTD) and from the beginning of the quarter (QTD). These functions can be passed as filters to the DAX CALCULATE function .

Function DAX TOTALYTD —evaluates the expression for YTD in the current filter context. Also included are the equivalent DAX functions for the QTD and MTD period TOTALQTD and TOTALMTD .

Function DAX DATESBETWEEN — returns a table containing a column with dates starting from the start date value and ending with the end date value.

Function DAX DATESINPERIOD — returns a table containing a date column that starts with the start date value and continues for the specified number of intervals..

METHODS FOR GENERATING SUMMARY DATA BY TIME

Function DAX TOTALYTD —evaluates the expression for YTD in the current filter context. Also included are the equivalent DAX functions for the QTD and MTD period TOTALQTD and TOTALMTD .

The syntax of TOTALYTD is as follows :

TOTALYTD(<expression>, <dates>, [, <filter>][, <year_end_date>])

The function requires an expression and a reference to the date column of the labeled date table. If necessary, you can pass a single filter criterion or the end date of the year (required only if the year does not end on December 31).



EXAMPLE

A measure definition that calculates revenue from the beginning of the year. Format the measure as a currency with two decimal places.

6. Revenue YTD =
TOTALYTD([Revenue],
'Date'[Date], "6-30")

Value of the end date of the year 6-30 corresponds to June 30 (Fiscal year).

Month	Revenue	Revenue YTD
2017 Jul	1 423 357,32 ₱	1 423 357,32 ₱
2017 Aug	2 057 902,45 ₱	3 481 259,78 ₱
2017 Sep	2 523 947,55 ₱	6 005 207,32 ₱
2017 Oct	561 681,48 ₱	6 566 888,80 ₱
2017 Nov	4 764 920,16 ₱	11 331 808,96 ₱
2017 Dec	596 746,56 ₱	11 928 555,52 ₱
2018 Jan	1 327 674,63 ₱	13 256 230,15 ₱
2018 Feb	3 936 463,31 ₱	17 192 693,45 ₱
2018 Mar	700 873,18 ₱	17 893 566,64 ₱
2018 Apr	1 519 275,24 ₱	19 412 841,88 ₱
2018 May	2 960 378,09 ₱	22 373 219,97 ₱
2018 Jun	1 487 671,19 ₱	23 860 891,17 ₱
2018 Jul	2 939 691,00 ₱	2 939 691,00 ₱
2018 Aug	3 964 801,20 ₱	6 904 492,20 ₱
2018 Sep	3 287 605,93 ₱	10 192 098,13 ₱
2018 Oct	2 157 287,40 ₱	12 349 385,53 ₱
Total	109 809 274,20 ₱	

RENAMING MEASURES FOR VISUALIZATIONS

The screenshot displays the Microsoft Power BI Desktop interface. The main area shows a table with three columns: 'Month', 'Revenue', and 'Revenue YTD'. The data spans from July 2017 to October 2018, with a total revenue of 109,809,274.20 P. The 'Visualizations' pane on the right is active, showing a list of fields including 'Fiscal Year', 'Full Date', 'Month', 'Product', 'Reseller', 'Sales', 'Sales Order', 'Sales Territory', and '1.Revenue'. A context menu is open over the '1.Revenue' field, with the option 'Rename for this visual' highlighted by a red circle. The 'Filters' pane on the left shows filters for 'Month', 'Revenue', and 'Revenue YTD'. The 'Fields' pane on the right shows a list of fields with checkboxes for selection. The bottom of the screen shows a navigation bar with 'Introduction', 'Page 1', 'Page 2', and 'Page 3'.

Month	Revenue	Revenue YTD
2017 Jul	1 423 357,32 P	1 423 357,32 P
2017 Aug	2 057 902,45 P	3 481 259,78 P
2017 Sep	2 523 947,55 P	6 005 207,32 P
2017 Oct	561 681,48 P	6 566 888,80 P
2017 Nov	4 764 920,16 P	11 331 808,96 P
2017 Dec	596 746,56 P	11 928 555,52 P
2018 Jan	1 327 674,63 P	13 256 230,15 P
2018 Feb	3 936 463,31 P	17 192 693,45 P
2018 Mar	700 873,18 P	17 893 566,64 P
2018 Apr	1 519 275,24 P	19 412 841,88 P
2018 May	2 960 378,09 P	22 373 219,97 P
2018 Jun	1 487 671,19 P	23 860 891,17 P
2018 Jul	2 939 691,00 P	2 939 691,00 P
2018 Aug	3 964 801,20 P	6 904 492,20 P
2018 Sep	3 287 605,93 P	10 192 098,13 P
2018 Oct	2 157 287,40 P	12 349 385,53 P
Total	109 809 274,20 P	

Calculating new occurrences

Another application of the logic functions of operations over time is the counting of new occurrences.

The following example shows how you can calculate the number of new customers for a specific time period. A new customer is counted for the period of time during which they made their first purchase.

The first task is to add the following measure definition, which counts the number of unique customers for the entire period (LTD).

For the entire period means from the start date to the end date in the filter context.



NEW MEASURE

7. Customers LTD =

VAR

```
CustomersLTD = CALCULATE (  
    DISTINCTCOUNT(Sales[CustomerKey]), DATESBETWEEN (  
        'Date'[Date],  
        BLANK(),  
        MAX('Date'[Date])  
    ),  
    'Sales Order'[Channel] = "Internet"  
)  
RETURN  
CustomersLTD
```

DATESBETWEEN(<dates>, <start_date>, <end_date>)

Month	Revenue	Revenue YTD	Customers LTD
			18484
2017 Jul	1 423 357,32 ₪	1 423 357,32 ₪	289
2017 Aug	2 057 902,45 ₪	3 481 259,78 ₪	448
2017 Sep	2 523 947,55 ₪	6 005 207,32 ₪	609
2017 Oct	561 681,48 ₪	6 566 888,80 ₪	783
2017 Nov	4 764 920,16 ₪	11 331 808,96 ₪	1013
2017 Dec	596 746,56 ₪	11 928 555,52 ₪	1201
2018 Jan	1 327 674,63 ₪	13 256 230,15 ₪	1394
2018 Feb	3 936 463,31 ₪	17 192 693,45 ₪	1571
2018 Mar	700 873,18 ₪	17 893 566,64 ₪	1790
2018 Apr	1 519 275,24 ₪	19 412 841,88 ₪	1992
2018 May	2 960 378,09 ₪	22 373 219,97 ₪	2214
2018 Jun	1 487 671,19 ₪	23 860 891,17 ₪	2459
2018 Jul	2 939 691,00 ₪	2 939 691,00 ₪	2740
2018 Aug	3 964 801,20 ₪	6 904 492,20 ₪	2946
2018 Sep	3 287 605,93 ₪	10 192 098,13 ₪	3167
Total	109 809 274,20 ₪		18484

The number of unique LTD clients has been counted by the end of each month.